Java configuration example

```
package org.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class App {
   public static void main(String[] args) {
      // Load the Java Config
      ApplicationContext context = new AnnotationConfigApplicationContext(javaconfiguration.class);

      // Get the bean
      Rahul obj = context.getBean("rahulBean", Rahul.class);

      // Call a method
      obj.study();

      Vishal objq=context.getBean("vishalbean ",Vishal.class);
      objq.study();
   }
}
```

```
package org.example;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class javaconfiguration {

   @Bean
   public Rahul rahulBean() {
      return new Rahul();
   }

   @Bean
   public Vishal vishalBean(){
      return new Vishal();
   }
}
```

```
package org.example;

public class Vishal {
   public void study(){
      System.out.println("he is studiing");
   }
}
```

```
package org.example;

public class Rahul { // Capitalized 'R' to match your App class
   public void study() {
      System.out.println("Rahul is now mastering Java Config!");
   }
}
```

To finish **Exercise 1**, you need to make sure your Car and Engine classes are marked so Spring can find them. In Spring Boot, this is done using the @Component annotation.

When you run SpringApplication.run, Spring Boot performs a **Component Scan**. It looks through your packages, finds anything with @Component, creates an instance (a "Bean"), and stores it in the **ApplicationContext** (the Container).

1. The Engine Bean

Create a file named Engine.java.

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class DemoApplication {
   public static void main(String[] args) {
      ApplicationContext context = SpringApplication.run(DemoApplication.class, args);

      // Fetch ONLY the Car
      Car car = context.getBean(Car.class);

      // Call drive() - it works because the engine was injected!
      car.drive();
   }
}
```

```
package com.example.demo.controller;


import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;

@Component
public class Engine {
   public void start() {
      System.out.println("Engine started... Vroom!");
   }
}
```

```
package com.example.demo;

import com.example.demo.controller.Engine;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Car {

   @Autowired // Field Injection
   private Engine engine;

   public void drive() {
```

```
        engine.start(); // Using the injected engine
        System.out.println("Car is moving smoothly!");
    }
}
```

Concepts Checklist

- **Spring Boot Startup**: SpringApplication.run starts the entire framework and the embedded server (if applicable).
- **IoC Container**: This is the ApplicationContext. It acts as a "bucket" that holds your objects so you don't have to manage them manually.
- **@SpringBootApplication**: This single annotation enables **Auto-Configuration** and **Component Scanning**.
- **@Component**: A generic stereotype for any Spring-managed component.

using constructor injection

1. The Updated Car Class

You will remove the @Autowired from the private field and move it to a constructor.

Note: In modern Spring Boot (4.3+), if a class has only **one** constructor, you don't even need to write the @Autowired annotation—Spring will assume you want to inject the parameters automatically!

Java

```
package com.example.demo;

import com.example.demo.controller.Engine;

import org.springframework.stereotype.Component;


@Component

public class Car {


    // 1. Mark the dependency as 'final' (Best Practice)

    private final Engine engine;

   // 2. Use a Constructor to inject the dependency

    public Car(Engine engine) {

        this.engine = engine;

        System.out.println("Car initialized via Constructor Injection!");

    }


    public void drive() {

        engine.start();

        System.out.println("Car is moving smoothly!");

    }
```

```
}
```

2. The Engine Class

This stays exactly as it was. Spring discovers it via @Component and prepares it to be passed into the Car constructor.

Java

```
package com.example.demo.controller;


import org.springframework.stereotype.Component;


@Component
public class Engine {
    public void start() {
        System.out.println("Engine started... Vroom!");
    }
}
```

3. The DemoApplication Class

Your main class remains unchanged. The way you *use* the bean doesn't change; only the way Spring *builds* the bean changes.

Java

```
package com.example.demo;


import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;


@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(DemoApplication.class, args);


        // Fetch the Car bean
        Car car = context.getBean(Car.class);


        // This works perfectly because Spring injected the Engine during construction
        car.drive();
    }
}
```

Why is Constructor Injection better?

1. **Mandatory Dependencies**: You can't create a Car object without providing an Engine. With Field Injection, you could accidentally create a "broken" car with a null engine.
2. **Immutability**: By using the final keyword, you guarantee the engine won't be swapped out for a different one later.
3. **Easy Testing**: You can test the Car class in a plain Java unit test by just doing new Car(new MockEngine()) without needing to start the whole Spring framework.

@Scope

1. The Configuration (JavaConfig.java)

To change the scope, you use the @Scope annotation. Replace your //todo with the following:

Java

```
package com.example.demo;


import com.example.demo.controller.Engine;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.context.annotation.Scope;


@Configuration

public class JavaConfig {


    @Bean

    @Scope("prototype") // <--- This is the magic line

    public Engine engine() {

        return new Engine();

    }

}
```

2. Testing in DemoApplication.java

To see the difference, print the objects themselves. Since Java prints the memory address (hashcode) of the object by default, different addresses mean different objects.

Java

```
public class DemoApplication {

    public static void main(String[] args) {

        ApplicationContext context = SpringApplication.run(DemoApplication.class, args);
```

```
        // Fetching the bean twice
        Engine e1 = context.getBean(Engine.class);
        Engine e2 = context.getBean(Engine.class);

        // Checking if they are the same instance
        System.out.println("Engine 1: " + e1);
        System.out.println("Engine 2: " + e2);

        System.out.println("Is same instance? " + (e1 == e2));
    }
}
```

---

3. Understanding the Output

Scope	Result of e1 == e2	Explanation
Singleton (Default)	true	Spring gives you the exact same object from its cache.
Prototype	false	Spring runs the new Engine() code every time you call getBean().

Also

```
package com.example.demo.controller;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;
@Component
@Scope("prototype") // Default is "singleton"
public class Engine {
    public Engine() {
        System.out.println(">> New Engine Instance Created!");
    }
}
```

This is a classic Spring "Ambiguity" problem. When you have one interface (Engine) and two implementations (PetrolEngine and DieselEngine), Spring gets confused. If you try to run this code as-is, Spring will throw a NoUniqueBeanDefinitionException because it doesn't know which engine to put in the Car.

To fix this, you have two main tools: **@Primary** and **@Qualifier**.

1. Using @Primary

Use this when you want one engine to be the "default" choice. If you don't specify anything else, Spring will always pick the Primary one.

Java

```
@Component
@Primary // This makes PetrolEngine the default choice
public class PetrolEngine implements Engine {
    public String start() {
        return "Petrol engine started";
    }
}
```

2. Using @Qualifier

Use this when you want to be very specific about which bean to inject in a particular class. This overrides @Primary.

The Car Class

Java

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class Car {
    private final Engine engine;

    // We use @Qualifier to tell Spring EXACTLY which bean name to use
    // The bean name is the class name starting with a lowercase letter
    public Car(@Qualifier("dieselEngine") Engine engine) {
        this.engine = engine;
    }

    public void drive() {
        System.out.println(engine.start());
    }
}
```

3. The Interface and Implementations

Here is the rest of your code to complete the setup:

Engine.java

Java

```
public interface Engine {
   String start();
}
```

DieselEngine.java

Java

```
@Component
public class DieselEngine implements Engine {
   public String start() {
      return "Diesel engine started... Chug chug!";
   }
}
```

Summary of Concepts

Annotation	Purpose	Analogy
@Primary	Sets a default bean.	"If I don't ask for a specific brand, give me the house brand."
@Qualifier	Selects a specific bean by name.	"I specifically want the 'Diesel' brand."
Interface Design	Allows you to swap engines without changing the Car code.	A standard plug that fits any wall socket.

Gemini said

This exercise introduces you to the **Standard Layered Architecture** (often called N-Tier Architecture). In Spring, we use specialized "stereotype" annotations to define the role of each class.

Even though @Component, @Service, @Repository, and @Controller all register beans in the container, using the specific ones adds extra features (like automatic exception translation in Repositories).

---

1. The Model (Car.java**)**

This is a simple POJO (Plain Old Java Object) representing your data.

Java

```
package com.example.demo.model;


public class Car {
```

```
    private String model;

    // Constructor, Getters, Setters

    public Car(String model) { this.model = model; }

    @Override

    public String toString() { return "Car: " + model; }

}
```

2. The Repository (CarRepository.java**)**

This layer communicates with the Database. We use @Repository.

Java

```
package com.example.demo.repository;


import com.example.demo.model.Car;

import org.springframework.stereotype.Repository;


@Repository

public class CarRepository {

    public Car findCarById() {

        // In a real app, this would be a DB query

        return new Car("Tesla Model 3");

    }

}
```

3. The Service (CarService.java**)**

This layer holds the **Business Logic**. It sits between the Controller and the Repository. We use @Service.

Java

```
package com.example.demo.service;


import com.example.demo.model.Car;

import com.example.demo.repository.CarRepository;

import org.springframework.stereotype.Service;


@Service

public class CarService {

    private final CarRepository carRepository;


    public CarService(CarRepository carRepository) {
```

```
        this.carRepository = carRepository;
    }

    public String getCarDetails() {
        Car car = carRepository.findCarById();
        return "Service processed: " + car.toString();
    }
}
```

4. The Controller (CarController.java**)**

This layer handles HTTP requests (Web Layer). We use @Controller or @RestController.

Java

```
package com.example.demo.controller;

import com.example.demo.service.CarService;
import org.springframework.stereotype.Controller;

@Controller
public class CarController {
    private final CarService carService;

    public CarController(CarService carService) {
        this.carService = carService;
    }

    public void display() {
        System.out.println(carService.getCarDetails());
    }
}
```

---

5. Testing the Flow (DemoApplication.java**)**

Java

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(DemoApplication.class, args);
```

```
        // We only call the Controller; it triggers the Service, which triggers the Repository
        CarController controller = context.getBean(CarController.class);
        controller.display();
    }
}
```

Summary of Stereotype Annotations

Annotation	Layer	Purpose
@Controller	Presentation	Handles Web/API requests and returns views/data.
@Service	Business	Contains logic, calculations, and coordinates data.
@Repository	Data Access	Encapsulates storage, retrieval, and search behavior.
@Component	General	Any other generic Spring-managed bean.

In **Exercise 10**, you are moving away from hardcoding strings in your Java code and moving them into a central configuration file: application.properties.

Spring Boot automatically reads this file and allows you to "inject" those values directly into your variables using the **@Value** annotation.

---

1. The Configuration (application.properties)

Located in src/main/resources/application.properties. This file acts as the "Settings" menu for your entire app.

Properties

```
# Change the default port (8080) to 9000
server.port=9000

# Custom property
car.brand=Toyota

# Spring internal property
```

```
spring.application.name=car-app
```

2. The Controller (CarController.java)

Use **@RestController** and **@GetMapping** to make this a real web endpoint.

Java

```
package com.example.demo.controller;


import org.springframework.beans.factory.annotation.Value;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;


@RestController

public class CarController {


    // Inject the value from application.properties

    @Value("${car.brand}")

    private String brand;


    @Value("${spring.application.name}")

    private String appName;


    // This creates the URL endpoint

    @GetMapping("/car")

    public String getCarBrand() {

        return "App: " + appName + " | Brand: " + brand;

    }

}
```

3. Testing the Result

1. **Run the application**: You will notice in the console that Tomcat now starts on **port 9000** instead of 8080.
2. **Open your browser**: Navigate to http://localhost:9000/car.
3. **Result**: You should see the text: App: car-app | Brand: Toyota.

Key Concepts Learned

- **@Value("${...}")**: This is a "Spring Expression" that looks up a key in your properties file. If the key is missing, the application will fail to start.
- **Default Values**: You can provide a fallback like this: @Value("${car.color:Red}"). If car.color isn't in the properties file, it will use "Red".

- **Server Configuration**: server.port is a built-in Spring Boot property that lets you avoid port conflicts with other apps.